BTS CIEL · PROGRAMMATION PHP · TRAVAUX PRATIQUES

TP3 — Authentification avec sessions + cookies

Cours associé : `01_Cours/Cours_Sessions_Cookies.pdf` *(sections 0 à 7).*

Prérequis : Cours PHP, sections 9 à 11 et 13 ; exercices 1 à 5 de Cookies & Sessions conseillés.

Démarche : ce TP est **guidé** — le code de chaque fichier est fourni. Il faut créer les fichiers, tester chaque fonctionnalité dans le navigateur et dans les outils de développement (onglets *Réseau* et *Cookies*), puis répondre aux questions. Les extensions E1 à E6 demandent ensuite de modifier ce code.

Objectif

Créer un système d'authentification complet :

1. Inscription
2. Connexion
3. « Se souvenir de moi »
4. Compteur de visites
5. Page privée
6. Déconnexion

Toujours **sans base de données** : les comptes sont conservés dans un **fichier JSON** (`data/users.json`).

Arborescence du dossier

```
/tp-final
├── index.php
├── register.php
├── login.php
├── logout.php
├── private.php
├── auth.php        (session, fonctions communes, reconnexion « se souvenir de moi »)
├── users.php       (stockage des comptes : simulateur de base de données)
├── header.php
├── footer.php
└── data/
    └── users.json  (créé automatiquement)
```

> **Règle d'or du TP** : tout le **traitement** (sessions, cookies, redirections) se fait **avant** `include "header.php"`, qui commence à afficher du HTML.

1. `users.php` — stockage des comptes

```php
<?php
const FICHIER_USERS = __DIR__ . "/data/users.json";

function chargerUsers(): array
{
    if (!is_file(FICHIER_USERS)) {
        return [];
    }
    $donnees = json_decode(file_get_contents(FICHIER_USERS), true);
    return is_array($donnees) ? $donnees : [];
}

function sauvegarderUsers(array $users): void
{
    if (!is_dir(dirname(FICHIER_USERS))) {
        mkdir(dirname(FICHIER_USERS), 0775, true);
    }
    file_put_contents(
        FICHIER_USERS,
        json_encode($users, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE),
        LOCK_EX
    );
}
```

Chaque compte est de la forme `"pseudo" => ["hash" => "...", "remember" => null]`.

> En production, un fichier de données ne doit pas être accessible par URL (le placer hors du dossier public, ou interdire l'accès).

2. `auth.php` — session et fonctions communes

```php
<?php
session_start([
    "cookie_httponly" => true,
    "cookie_samesite" => "Lax",
    "use_strict_mode" => true,
]);

require_once __DIR__ . "/users.php";

/** Échappe une valeur pour l'affichage HTML. */
function e(string $texte): string
{
    return htmlspecialchars($texte, ENT_QUOTES, "UTF-8");
}

/** Envoie un cookie avec des options sûres. */
function poserCookie(string $nom, string $valeur, int $expire): void
{
    setcookie($nom, $valeur, [
        "expires"  => $expire,
        "path"     => "/",
        "httponly" => true,
        "samesite" => "Lax",
    ]);
}

// Reconnexion automatique grâce au cookie « se souvenir de moi »
if (!isset($_SESSION["user"]) && is_string($_COOKIE["remember"] ?? null)) {
    [$pseudo, $jeton] = array_pad(explode(":", $_COOKIE["remember"], 2), 2, "");
    $users   = chargerUsers();
    $attendu = $users[$pseudo]["remember"] ?? null;

    if ($attendu !== null && hash_equals($attendu, hash("sha256", $jeton))) {
        session_regenerate_id(true);
        $_SESSION["user"] = $pseudo;
    }
}
```

3. `index.php` — accueil et compteur de visites (cookie)

```php
<?php
require "auth.php";

// Compteur de visites via cookie : la valeur reçue est validée
$visites = filter_var($_COOKIE["visites"] ?? 0, FILTER_VALIDATE_INT, ["options" =>
["min_range" => 0]]);
$visites = ($visites === false ? 0 : $visites) + 1;

poserCookie("visites", (string) $visites, time() + 3600 * 24);   // 24 h

$titre = "Accueil";
include "header.php";
?>

<h1>Bienvenue</h1>

<p>Nombre de visites : <strong><?= $visites ?></strong></p>

<?php include "footer.php"; ?>
```

4. `register.php` — inscription

```php
<?php
require "auth.php";

$erreurs = [];
$succes  = false;
$pseudo  = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $pseudo = trim($_POST["pseudo"] ?? "");
    $pass   = $_POST["password"] ?? "";          // pas de trim() sur un mot de passe

    if (!preg_match('/^[A-Za-z0-9_]{3,20}$/', $pseudo)) {
        $erreurs[] = "Pseudo : 3 à 20 caractères (lettres, chiffres, _).";
    }
    if (!preg_match('/^(?=.*[A-Z])(?=.*\d).{8,}$/', $pass)) {
        $erreurs[] = "Mot de passe : 8 caractères minimum, une majuscule et un chiffre.";
    }

    $users = chargerUsers();
    if (isset($users[$pseudo])) {
        $erreurs[] = "Ce pseudo est déjà utilisé.";
    }

    if (empty($erreurs)) {
        $users[$pseudo] = [
            "hash"     => password_hash($pass, PASSWORD_DEFAULT),
            "remember" => null,
        ];
        sauvegarderUsers($users);
        $succes = true;
        $pseudo = "";
    }
}

$titre = "Inscription";
include "header.php";
?>

<h1>Inscription</h1>

<?php if (!empty($erreurs)) : ?>
    <ul style="color: red;">
        <?php foreach ($erreurs as $erreur) : ?>
            <li><?= e($erreur) ?></li>
        <?php endforeach; ?>
    </ul>
<?php endif; ?>

<?php if ($succes) : ?>
    <p style="color: green;">Inscription réussie ! Vous pouvez vous <a
href="login.php">connecter</a>.</p>
<?php endif; ?>

<form method="post">
    <label for="pseudo">Pseudo :</label>
    <input type="text" id="pseudo" name="pseudo" value="<?= e($pseudo) ?>"><br><br>

    <label for="password">Mot de passe :</label>
    <input type="password" id="password" name="password"><br><br>

    <button type="submit">S'inscrire</button>
```

```
</form>

<?php include "footer.php"; ?>
```

5. `login.php` — connexion et « se souvenir de moi »

```php
<?php
require "auth.php";

$erreurs = [];
$pseudo  = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $pseudo   = trim($_POST["pseudo"] ?? "");
    $pass     = $_POST["password"] ?? "";
    $remember = isset($_POST["remember"]);

    $users = chargerUsers();
    $hash  = $users[$pseudo]["hash"] ?? null;

    if ($hash !== null && password_verify($pass, $hash)) {
        session_regenerate_id(true);
        $_SESSION["user"] = $pseudo;

        if ($remember) {
            // Jeton aléatoire : le serveur ne garde que son empreinte
            $jeton = bin2hex(random_bytes(32));
            $users[$pseudo]["remember"] = hash("sha256", $jeton);
            sauvegarderUsers($users);
            poserCookie("remember", $pseudo . ":" . $jeton, time() + 3600 * 24 * 7);   // 7
jours
        }

        header("Location: private.php");
        exit;
    }

    $erreurs[] = "Identifiants incorrects.";      // message générique
}

$titre = "Connexion";
include "header.php";
?>

<h1>Connexion</h1>

<?php if (!empty($erreurs)) : ?>
    <ul style="color: red;">
        <?php foreach ($erreurs as $erreur) : ?>
            <li><?= e($erreur) ?></li>
        <?php endforeach; ?>
    </ul>
<?php endif; ?>

<form method="post">
    <label for="pseudo">Pseudo :</label>
    <input type="text" id="pseudo" name="pseudo" value="<?= e($pseudo) ?>"><br><br>

    <label for="password">Mot de passe :</label>
    <input type="password" id="password" name="password"><br><br>

    <input type="checkbox" id="remember" name="remember">
    <label for="remember">Se souvenir de moi</label><br><br>

    <button type="submit">Se connecter</button>
</form>
```

```php
<?php include "footer.php"; ?>
```

6. `private.php` — page protégée

La page n'est accessible que si l'utilisateur est connecté, par la session **ou** par le cookie « se souvenir de moi » (géré dans `auth.php`).

```php
<?php
require "auth.php";

if (!isset($_SESSION["user"])) {
    http_response_code(403);
    $titre = "Accès interdit";
    include "header.php";
    echo "<h1>Accès interdit</h1>";
    echo "<p>Vous devez être connecté.</p>";
    echo '<a href="login.php">Connexion</a>';
    include "footer.php";
    exit;
}

$titre = "Espace privé";
include "header.php";
?>

<h1>Espace privé</h1>

<p>Bienvenue, <strong><?= e($_SESSION["user"]) ?></strong> !</p>

<p><a href="logout.php">Déconnexion</a></p>

<?php include "footer.php"; ?>
```

7. `logout.php` — déconnexion complète

Supprime la session, le cookie de session, le cookie « remember » **et** le jeton côté serveur.

```php
<?php
require "auth.php";

if (isset($_SESSION["user"])) {
    $users = chargerUsers();
    if (isset($users[$_SESSION["user"]])) {
        $users[$_SESSION["user"]]["remember"] = null;      // révoque le jeton
        sauvegarderUsers($users);
    }
}

$_SESSION = [];
setcookie(session_name(), "", ["expires" => time() - 3600, "path" => "/"]);
setcookie("remember", "", ["expires" => time() - 3600, "path" => "/"]);
session_destroy();

header("Location: index.php");
exit;
```

8. `header.php` et `footer.php`

`header.php`

```php
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title><?= e($titre ?? "TP Cookies/Sessions") ?></title>
</head>
<body>
<nav>
    <a href="index.php">Accueil</a>
    <?php if (isset($_SESSION["user"])) : ?>
        | <a href="private.php">Espace privé</a>
        | <a href="logout.php">Déconnexion (<?= e($_SESSION["user"]) ?>)</a>
    <?php else : ?>
        | <a href="register.php">Inscription</a>
        | <a href="login.php">Connexion</a>
    <?php endif; ?>
</nav>
<hr>
```

`footer.php`

```php
</body>
</html>
```

Questions

1. Pourquoi `setcookie()` et `header("Location: ...")` sont-ils appelés **avant** `include "header.php"` ?
2. Pourquoi les comptes sont-ils stockés dans un fichier et non dans un tableau PHP ?
3. Pourquoi le cookie « remember » contient-il un jeton aléatoire et non le pseudo seul ?
4. Pourquoi `session_regenerate_id(true)` est-il appelé à chaque ouverture de session ?
5. Pourquoi le serveur ne stocke-t-il que l'**empreinte** du jeton ?
6. Pourquoi le message d'erreur de connexion est-il identique pour un pseudo inconnu et un mot de passe faux ?
7. Pourquoi ne fait-on pas `trim()` sur le mot de passe ?

Limites de cette version (à discuter)

Le code est volontairement simple. En situation réelle, on irait plus loin :

- **un seul jeton « se souvenir de moi » par compte** : se connecter avec la case cochée sur un second navigateur remplace le jeton et déconnecte le premier navigateur à sa prochaine visite. Une vraie application stocke une liste de jetons (un par appareil) ;
- **le jeton n'est pas renouvelé** après une reconnexion automatique : un jeton volé reste valable jusqu'à son expiration ou jusqu'à la déconnexion ;
- `data/users.json` **est dans le dossier public** : il serait téléchargeable par URL (empreintes des mots de passe et des jetons). Il faut le placer hors de `htdocs` ou en interdire l'accès ;

- `auth.php` **crée des variables globales** (`$pseudo` , `$jeton` , `$users`) visibles dans les pages qui l'incluent : les pages les réinitialisent avant usage, mais il serait plus propre d'isoler ce code dans une fonction.

Extensions

Ces exercices modifient le code du TP : les réaliser une fois le TP terminé et testé.

E1 — Forger un cookie

Sur ce TP, remplacer temporairement la gestion de « se souvenir de moi » par la **version dangereuse** décrite à la section 6 du cours (cookie `remember = pseudo`). Se connecter en tant qu'administrateur **sans connaître son mot de passe** en créant le cookie à la main. Expliquer l'attaque, puis rétablir la version sécurisée et vérifier que l'attaque échoue.

E2 — Dernière connexion

Mémoriser dans un cookie `last_login` la date de connexion, et afficher dans `private.php` la date de la **connexion précédente** (pas celle en cours).

E3 — Oublier ce navigateur

Ajouter dans `private.php` un bouton « Ne plus me reconnaître sur ce navigateur » qui supprime le cookie `remember` **et** révoque le jeton côté serveur, **sans** déconnecter l'utilisateur.

E4 — Compteur de visites sur 10 minutes

Réduire la durée de vie du cookie `visites` à 10 minutes. Que devient le compteur si l'on revient 15 minutes plus tard ?

E5 — Bloquer la connexion après 3 essais ratés

Après 3 échecs de connexion, bloquer les nouvelles tentatives pendant 5 minutes (compteur en session). Pourquoi cette protection est-elle **contournable** ? Comment la rendre réellement efficace ?

E6 — Thème clair/sombre

Ajouter deux liens « Clair » / « Sombre » qui mémorisent le choix dans un cookie `theme` . Appliquer le thème à toutes les pages (classe sur `<body>`). La valeur du cookie doit être **limitée aux valeurs autorisées** (`clair` , `sombre`).